ADR-001: Desain Struktur Class Domain Akademik dan Relasi Entitas

- **Status:** Diterima (*Accepted*)
- **Tanggal:** 2026-11-01
- **Penyusun:** Arsitek Sistem (System Architect)
- **Konteks Sistem:** Platform Akademik Kampus (*Academic Information System / SIAKAD Mini*)

---

1. Konteks & Latar Belakang (Context)

Sistem Informasi Akademik memerlukan fondasi pemodelan objek yang merepresentasikan siklus hidup civitas akademika:
* Berbagai peran pengguna (`Mahasiswa` , `Dosen` , `Admin` , `CalonMahasiswa`).
* Penyelenggaraan mata kuliah (`Course`) beserta dependensi kurikulum (`prasyarat`).
* Transaksi perkuliahan/registrasi mahasiswa pada mata kuliah (`Enrollment`).
* Penjadwalan alokasi ruang dan waktu perkuliahan (`Schedule`).
* Penilaian capaian belajar mahasiswa (`Grade`).

Tantangan arsitektur meliputi:
1. Menghindari keterikatan ketat (*tight coupling*) antar entitas inti.
2. Mematuhi prinsip **SOLID** agar sistem mudah diuji (*testable*) dan mudah diekstensi di masa mendatang.
3. Membedakan secara presisi kepemilikan siklus hidup data melalui relasi **Composition** vs **Aggregation**.

---

2. Keputusan Arsitektur & Struktur Class (Architectural Decision)

2.1 Struktur Class & Tanggung Jawab

Entitas / Interface	Tanggung Jawab Tunggal (*Single Responsibility*)	Rationale Desain
`User` **(Abstract)**	Menangani identitas dasar, kredensial autentikasi, dan audit waktu.	Basis polimorfisme untuk seluruh jenis pengguna sistem. Mengimplementasikan `IAuthenticatable`.
`Mahasiswa`	Mengelola profil akademik mahasiswa, IPK, riwayat KRS/KHS.	Turunan langsung dari `User`. Bertindak juga sebagai `IEnrollmentObserver` untuk merespons pembaruan nilai & status KRS.
`Dosen`	Mengelola profil staf pengajar, penginputan nilai, dan jadwal ajar.	Turunan langsung dari `User`. Berinteraksi dengan `Enrollment` saat pengisian nilai.
`Admin`	Manajemen akses user dan operasional kurikulum.	Turunan dari `User` dengan kapabilitas lintas modul (*cross-cutting operations*).
`CalonMahasiswa`	Mengelola pendaftaran mahasiswa baru, jalur masuk, dan seleksi berkas.	Subclass `User` dengan *lifecycle* spesifik sebelum resmi memperoleh NIM dan status `Mahasiswa`.
`Course`	Katalog mata kuliah, SKS, dan validasi kelayakan prasyarat.	Menjadi wadah kurikulum. Memiliki relasi *self-referential* untuk prasyarat dan mengelola jadwal perkuliahan.
`Schedule`	Alokasi hari, jam, ruangan, dan verifikasi konflik bentrok (*room conflict*).	Dipisahkan dari `Course` agar pemetaan alokasi ruang dan waktu tidak mengotori metadata kurikulum mata kuliah.
`Enrollment`	Entitas asosiatif transaksi perkuliahan antara `Mahasiswa` dan `Course`.	Menjadi *Aggregate Root* transaksi KRS. Mencegah kopling langsung *many-to-many* antara `Mahasiswa` dan `Course`.
`Grade`	Komponen nilai tugas, UTS, UAS, kalkulasi nilai akhir, dan huruf mutu.	Komponen nilai yang terikat hidup-matinya pada satu `Enrollment`.
`Payment`	Mengelola tagihan biaya perkuliahan (SPP/UKT), verifikasi pelunasan, dan audit transaksi keuangan per semester.	Menjadi syarat prasyarat finansial (*financial prerequisite*) sebelum pendaftaran/pengisian KRS (`Enrollment`) dapat divalidasi.

2.2 Keputusan Pemilihan Relasi (Composition vs Aggregation)

A. Composition (`*--`) — Relasi Kuat / *Lifecycle-bound*

1. `Enrollment` **$\rightarrow$** `Grade` :
2. **Alasan:** Nilai akademik (`Grade`) tidak memiliki eksistensi mandiri tanpa adanya transaksi pendaftaran kuliah (`Enrollment`). Apabila sebuah enrollment dibatalkan/dihapus (*hard delete* sebelum masa perkuliahan), record nilai tidak memiliki konteks bisnis yang valid dan wajib ikut terhapus (*cascade delete*).
3. `Course` **$\rightarrow$** `Schedule` :
4. **Alasan:** Jadwal ruang dan jam pertemuan secara semantik dimiliki oleh sesi mata kuliah yang bersangkutan. Menghapus penawaran mata kuliah pada semester tertentu secara otomatis melenyapkan alokasi jadwal tersebut.

B. Aggregation (`o--`) — Relasi Lemah / *Independent Lifecycle*

1. `Course` **$\rightarrow$** `Course` **(Self-referential Prerequisite)**:
2. **Alasan:** Mata kuliah prasyarat (misal: *Algoritma & Pemrograman*) tetap berdiri sendiri di basis data kurikulum meskipun mata kuliah tingkat lanjutannya (misal: *Struktur Data*) dihapus atau direvisi kurikulumnya.
3. `Mahasiswa` **$\rightarrow$** `Enrollment` :
4. **Alasan:** Mahasiswa tetap eksis sebagai entitas civitas akademika meskipun belum atau tidak melakukan KRS pada semester tertentu.
5. `Course` **$\rightarrow$** `Enrollment` :
6. **Alasan:** Mata kuliah tetap tercantum di katalog program studi meskipun pada semester berjalan tidak ada mahasiswa yang mendaftar.
7. `Dosen` **$\rightarrow$** `Course` :
8. **Alasan:** Penugasan dosen bersifat dinamis; dosen dapat cuti atau berpindah penugasan tanpa mematikan entitas mata kuliah.
9. `Mahasiswa` **$\rightarrow$** `Payment` :
10. **Alasan:** Mahasiswa tetap eksis dan riwayat pembayaran memiliki siklus hidup audit keuangan independen; pembatalan/revisi KRS tidak menghapus catatan transaksi finansial mahasiswa.

C. Dependency (`..>`) — Prasyarat Aturan Bisnis (*Business Rule Prerequisite*)

1. `Enrollment` **$\rightarrow$** `Payment` :
2. **Alasan:** Mahasiswa hanya dapat mengisi dan mengesahkan rencana studi (`Enrollment` / KRS) jika tagihan pembayaran (`Payment`) pada semester berjalan telah terverifikasi lunas (`isLunas() == true`). `Enrollment` memvalidasi kelayakan mahasiswa berdasarkan status objek `Payment` .

---

3. Penerapan Prinsip SOLID

- **Single Responsibility Principle (SRP):**
- `Schedule` bertanggung jawab atas ruang dan waktu (deteksi bentrok ruang).
- `Grade` fokus pada struktur komponen angka dan delegasi perhitungan nilai.
- `Enrollment` fokus mengontrol alur status pendaftaran (Pending, Approved, Dropped).
- `Payment` mengisolasi domain transaksi finansial, nominal, kanal bayar, dan audit status pelunasan terpisah dari domain akademik perkuliahan.
- **Open/Closed Principle (OCP):**
- Penambahan jenis peran baru (misalnya `Tendik` atau `Auditor`) cukup menambahkan subclass baru dari `User` tanpa mengubah logika otentikasi.
- Skema perhitungan nilai terbuka untuk penambahan strategi baru (misal kelas praktikum vs kelas teori) melalui `IGradeEvaluator`.
- **Liskov Substitution Principle (LSP):**
- Setiap subclass dari `User` dapat disubstitusikan ke dalam komponen otentikasi `IAuthenticatable` tanpa mengubah kebenaran program.
- **Interface Segregation Principle (ISP):**
- Antarmuka dipecah spesifik: `IAuthenticatable` hanya untuk akses login/logout, `IGradeEvaluator` untuk kalkulasi nilai, dan `IEnrollmentObserver` untuk notifikasi event perkuliahan. Tidak ada kelas yang dipaksa mengimplementasikan method yang tidak diperlukannya.
- **Dependency Inversion Principle (DIP):**
- `Enrollment` dan `Grade` bergantung pada abstraksi `IGradeEvaluator` daripada implementasi algoritma penilaian konkret.

---

4. Design Pattern yang Diterapkan

4.1 Factory Pattern (`UserFactory`)

- **Masalah:** Instansiasi user memerlukan validasi atribut spesifik per peran (NIM untuk Mahasiswa, NIDN untuk Dosen, nomor registrasi untuk Calon Mahasiswa).
- **Solusi:** `UserFactory` menyediakan method statis/kreasional `createUser(role, payload)` yang mengenkapsulasi pembuatan objek polimorfik `User`.
- **Manfaat:** Mengisolasi dependensi pembuatan objek dari modul pemanggil (controller / service layer).

4.2 Strategy Pattern (`IGradeEvaluator` , `StandardGradeEvaluator` , `PracticumGradeEvaluator`)

- **Masalah:** Bobot perhitungan nilai akhir bervariasi antara mata kuliah teori (Tugas 20%, UTS 30%, UAS 50%) dan mata kuliah praktikum/laboratorium (Praktikum 40%, Tugas 20%, Proyek Akhir

40%).

- **Solusi:** `Grade` menerima instansiasi `IGradeEvaluator`. Kelas `StandardGradeEvaluator` dan `PracticumGradeEvaluator` mengimplementasikan formula penilaian yang berbeda.
- **Manfaat:** Formula nilai dapat diubah atau ditukar secara *runtime* tanpa perlu mengubah class `Grade` atau class `Enrollment`.

4.3 Observer Pattern (`IEnrollmentObserver`)

- **Masalah:** Perubahan status akademik (misalnya saat Dosen menginput/memvalidasi nilai, atau saat persetujuan KRS oleh Dosen Wali) memerlukan pemicuan notifikasi ke Mahasiswa dan pembaruan IPK secara otomatis.
- **Solusi:** `Enrollment` bertindak sebagai *Subject/Observable* yang menyimpan daftar `IEnrollmentObserver`. `Mahasiswa` mengimplementasikan `IEnrollmentObserver`.
- **Manfaat:** Dekopel antara proses bisnis transaksi nilai dan mekanisme notifikasi/kalkulasi ulang IPK.

---

5. Pertimbangan & Trade-off (Trade-offs & Consequences)

5.1 Inheritance vs Role-based Composition pada `User`

- **Pilihan yang Diambil:** Pewarisan (*Inheritance*) dari abstract class `User`.
- **Kelebihan:**
- Kode sederhana, representasi mental UML yang sangat bersih dan mudah dimengerti.
- Tipe data kuat (*strong typing*) di level kompilasi (`Mahasiswa` berbeda tipe dengan `Dosen`).
- **Trade-off / Konsekuensi:**
- Kurang fleksibel jika seorang individu memiliki peran ganda secara simultan (misalnya: seorang Dosen yang juga mengambil studi S2/S3 sebagai Mahasiswa).
- **Mitigasi:** Jika di masa depan sistem memerlukan multi-peran pada 1 identitas fisik (Single Identity Multi-Role), arsitektur dapat berevolusi ke pola *Composition over Inheritance* menggunakan *Role/Party Pattern* (`User has-many Roles`).

5.2 Direct Association vs Associative Entity (`Enrollment`)

- **Pilihan yang Diambil:** Menggunakan `Enrollment` sebagai entitas asosiatif eksplisit antara `Mahasiswa` dan `Course`.
- **Kelebihan:**
- Memberikan tempat alami untuk menyimpan metadata relasional seperti `tanggalDaftar`, `statusApproval`, semester akademik, dan kepemilikan nilai `Grade`.
- **Trade-off / Konsekuensi:**
- Menambah jumlah class dan kompleksitas traversal relasi dibandingkan relasi langsung array/list `mahasiswa.courses`.

- **Justifikasi:** Dalam domain akademik riil, data tidak pernah murni *many-to-many* tanpa atribut transaksi. `Enrollment` adalah model wajib untuk menjamin integritas KRS.

5.3 Komposisi Jadwal Langsung pada `Course` vs `CourseOffering/Section`

- **Pilihan yang Diambil:** `Course` mengalokasikan langsung `Schedule` (sesuai spesifikasi kebutuhan ringkas).
- **Trade-off / Konsekuensi:**
- Pada skala kampus besar, sebuah mata kuliah dapat memiliki banyak kelas paralel (Kelas A, Kelas B) pada semester yang sama.
- **Mitigasi Masa Depan:** Jika sistem berkembang mendukung multi-kelas paralel per mata kuliah per semester, class `CourseOffering` / `KelasKuliah` dapat disisipkan di antara `Course` dan `Schedule` tanpa merombak prinsip relasi `Enrollment`.

---

6. Riwayat Perubahan & Evolusi Class Diagram (Architecture Evolution & Changelog)

6.1 Latar Belakang Perubahan

Pada iterasi awal perancangan (Baseline v1.0), sistem hanya memodelkan transaksi akademik murni (`Mahasiswa` , `Course` , `Enrollment` , `Grade`). Berdasarkan evaluasi proses bisnis institusi akademik, ditemukan celah krusial di mana pengisian KRS tidak memiliki validasi prasyarat pembayaran, sehingga mahasiswa berpotensi melakukan pendaftaran rencana studi tanpa menyelesaikan kewajiban finansial semester berjalan.

Oleh karena itu, pada iterasi revisi (v1.1):
1. Ditambahkan class inti `Payment` untuk mengelola tagihan biaya kuliah (SPP/UKT) dan verifikasi pelunasan.
2. Ditambahkan method validasi pada `Mahasiswa` (`isiKRS()` , `cekStatusPembayaran()` , `lihatRiwayatPembayaran()`).
3. Ditambahkan method validasi prasyarat finansial pada `Enrollment` (`validasiSyaratPembayaran(payment: Payment)`).
4. Ditambahkan method verifikasi pembayaran pada `Admin` (`verifikasiPembayaran(payment: Payment)`).
5. Ditambahkan relasi agregasi `Mahasiswa "1" o-- "0..*" Payment : pays` dan relasi dependensi `Enrollment ..> Payment : validates prerequisite` .

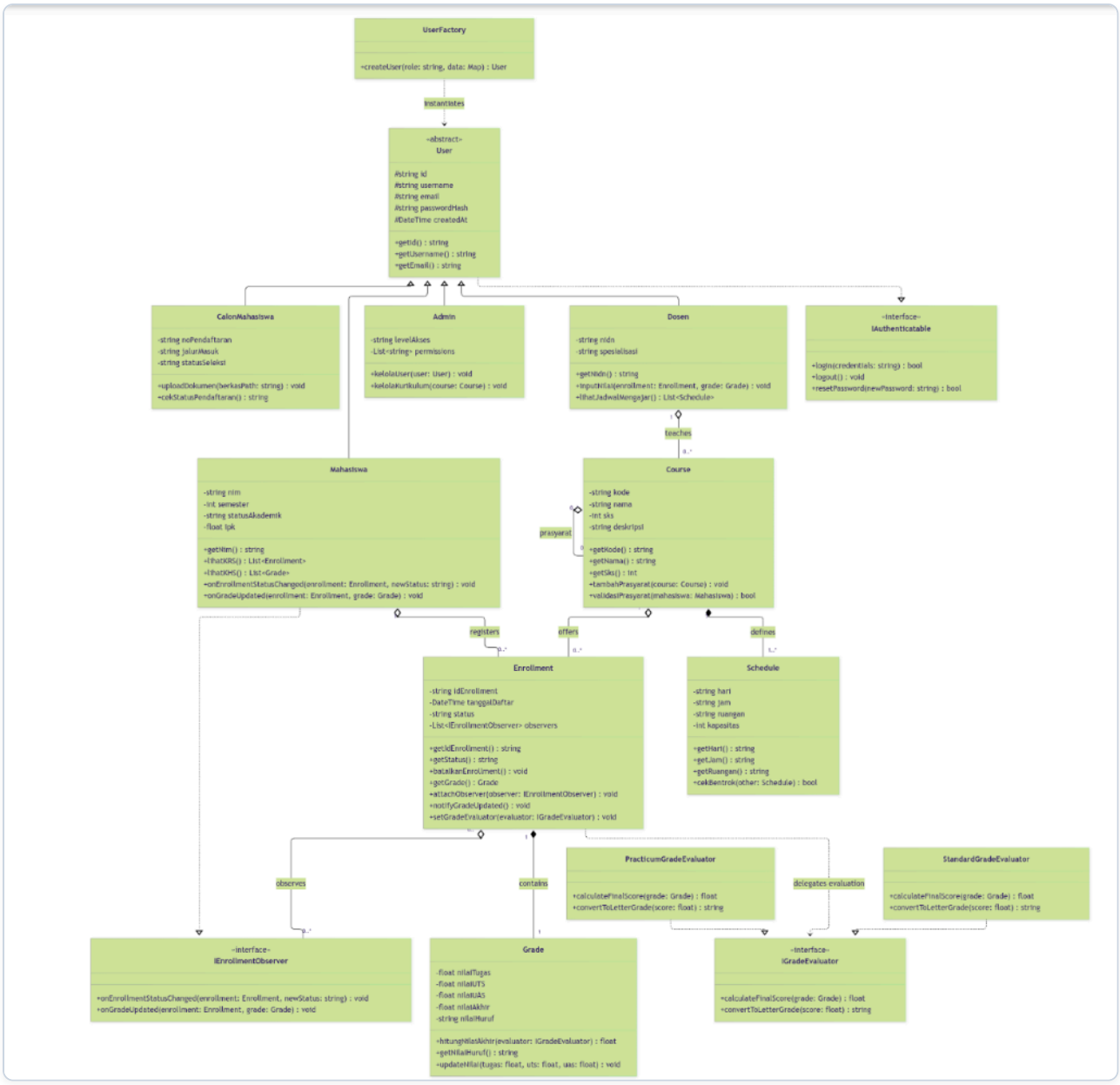
6.2 Matriks Komparasi Struktural (Before vs After)

Aspek / Komponen	Versi Awal (Baseline v1.0)	Versi Revisi (Dengan Payment v1.1)	Rationale & Dampak Desain
Entitas Finansial	Belum ada entitas pembayaran.	Ditambahkan class `Payment` dengan atribut tagihan & pelunasan.	Menjaga prinsip SRP dengan memisahkan domain audit finansial dari kurikulum.
Class `Mahasiswa`	Hanya melihat riwayat (`lihatKRS` , `lihatKHS`).	Ditambahkan `isiKRS()` , `cekStatusPembayaran()` , `lihatRiwayatPembayaran()` .	Mahasiswa dapat memvalidasi kelayakan tagihan sebelum mengirim draft KRS.
Class `Enrollment`	Pendaftaran tidak memvalidasi tagihan.	Ditambahkan `validasiSyaratPembayaran(payment: Payment) bool` .	Menjamin integritas bisnis: KRS tidak dapat disahkan tanpa status lunas.
Class `Admin`	Hanya mengelola user & kurikulum.	Ditambahkan `verifikasiPembayaran(payment: Payment) void` .	Memberikan staf administrasi/keuangan otoritas verifikasi pembayaran manual.
Relasi Finansial	Tidak ada relasi finansial.	`Mahasiswa "1" o-- "0..*" Payment` & `Enrollment ..> Payment` .	Pemodelan agregasi siklus hidup audit dan dependensi aturan bisnis KRS.

6.3 Dokumentasi Visual Perbandingan (Diagram Screenshots)

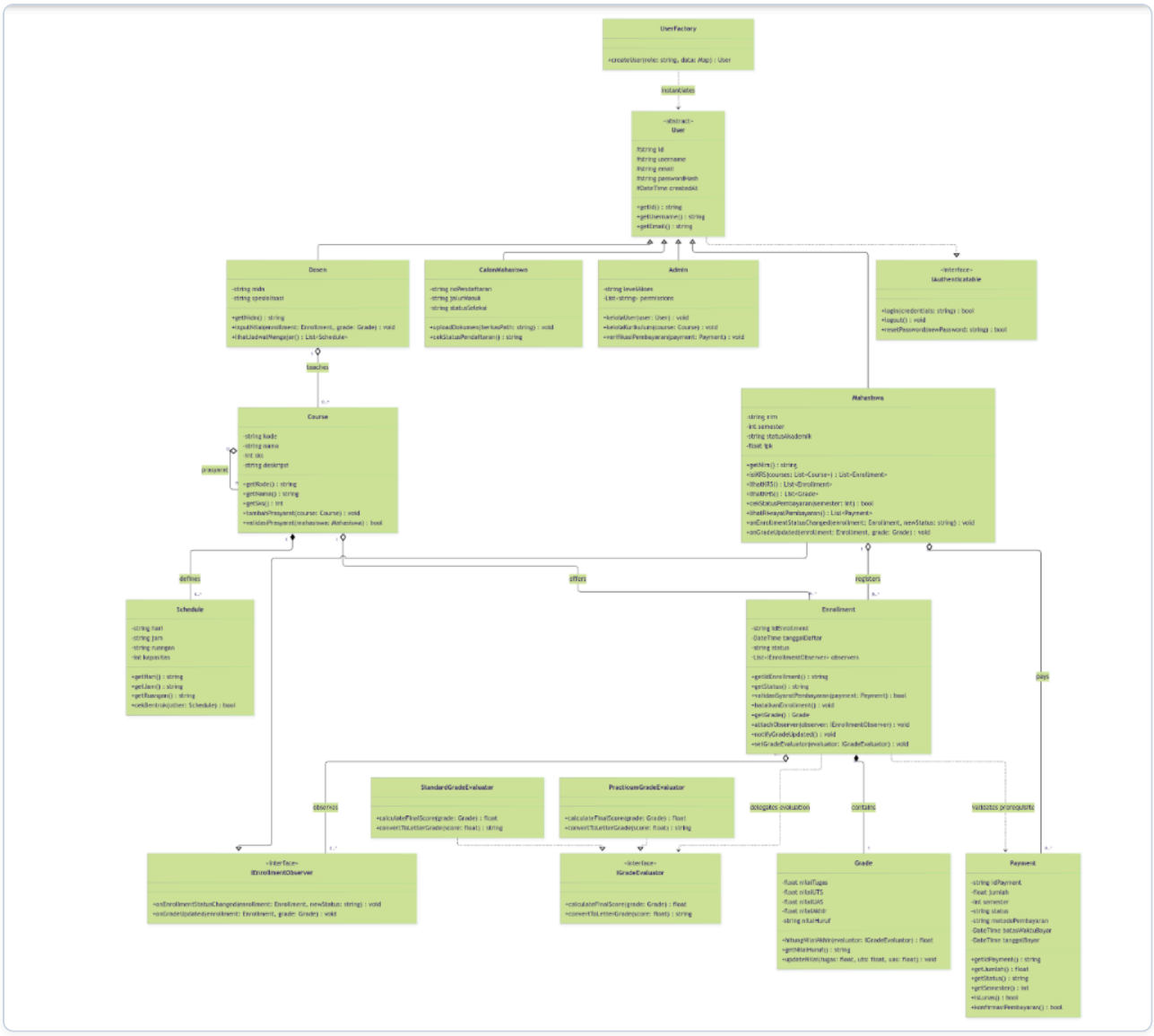
A. Versi Awal (Baseline v1.0 - Tanpa Entitas Payment)

Pada diagram awal, alur pendaftaran perkuliahan (`Enrollment`) hanya mengaitkan `Mahasiswa` dengan `Course` secara langsung tanpa adanya lapisan verifikasi finansial.



B. Versi Revisi (Revisi v1.1 - Dengan Entitas Payment & Prasyarat KRS)

Pada diagram revisi, entitas `Payment` ditambahkan di sudut kanan bawah dengan relasi kepemilikan oleh `Mahasiswa` (*Aggregation*) dan menjadi dependensi prasyarat mutlak bagi `Enrollment` (*Dependency*).



7. Referensi Berkas Terkait

- Diagram UML Mermaid Utama: `academic_class_diagram.mmd`
- Vektor Diagram SVG: `diagram.svg`
- Screenshot Baseline v1.0: `class-diagram-v1-baseline.png`
- Screenshot Revisi v1.1: `class-diagram-v2-payment.png`

Diagram Vektor: UML Class Diagram (SIAKAD Mini)

